Cal Poly Pomona

FALL 2026 POINTS: 100

Computer Science 4800

Software Engineering

Assignment #3

Agile Development

Objective

In this assignment, your group will plan, develop, and demonstrate a software product using Agile development practices. Your team will organize its work in either GitHub Projects or Jira and conduct regular Scrum meetings. The purpose of this assignment is to practice collaborative software development, iterative planning, task management, communication, and reflection.

Learning Objectives

By completing this assignment, you will be able to:

- Apply Scrum roles, events, and artifacts to a software project.
- Translate product requirements into user stories and development tasks.
- Estimate, prioritize, and track work using GitHub Projects or Jira.
- Collaborate through branches, pull requests, reviews, and issue tracking.
- Evaluate team progress using sprint reviews and retrospectives.
- Respond to changing requirements through iterative development.

Team Structure

Each team must identify the following responsibilities:

- **Product Owner**: Maintains and prioritizes the product backlog and clarifies requirements.
- **Scrum Master**: Facilitates Scrum meetings and helps address obstacles.

- **Developers**: Design, implement, test, review, and document the product.

The Product Owner and Scrum Master roles may rotate between sprints. Every student must contribute to development work.

Project Requirements

Your group will design and implement a software application approved by the instructor.
The project must include:

- A clearly stated product goal.
- At least two completed sprints.
- A prioritized product backlog.
- User stories with acceptance criteria.
- Assigned and estimated development tasks.
- Source code maintained in a shared GitHub repository.
- Meaningful commits from every team member.
- Feature branches and pull requests.
- At least one peer review for each major feature.
- Automated or documented testing.
- A working product increment at the end of each sprint.

Agile Project Board

Create a shared board using either GitHub Projects or Jira. Invite every team member and the instructor, if requested. At minimum, the board must contain these workflow stages:

1. Backlog
2. Ready
3. In Progress
4. In Review or Testing
5. Done

Each backlog item must include:

- A descriptive title.
- A user story or task description.
- Acceptance criteria.
- Priority.
- Estimate or story-point value.
- Assigned team member or members.
- Current status.
- Relevant links to code, pull requests, or documentation.

A story may only move to Done when its acceptance criteria are satisfied, its changes have been reviewed, and its tests have passed.

Scrum Activities

1. Sprint Planning

At the beginning of each sprint, your group must:

- Establish a sprint goal.
- Review and prioritize the product backlog.
- Select stories that the team can reasonably complete.
- Break large stories into manageable tasks.
- Estimate the selected work.
- Assign or volunteer for initial tasks.

Record the sprint goal and selected backlog items in GitHub Projects or Jira.

2. Scrum Meetings

Conduct regular Scrum meetings during each sprint. Meetings should be approximately 10–15 minutes. Each student must address:

- What have I completed since the previous meeting?

- What will I work on next?
- What obstacles or risks are affecting my work?

The Scrum Master must record:

- Meeting date and attendees.
- A short progress summary.
- Identified obstacles.
- Agreed follow-up actions.

Scrum meetings are for coordination rather than extended problem-solving. Schedule a separate discussion when an issue requires significant technical work.

3. Sprint Review

At the end of each sprint, demonstrate the completed product increment. Compare the results with the sprint goal and each story's acceptance criteria.
Record:

- Stories completed.
- Stories not completed.
- Demonstration results or stakeholder feedback.
- Backlog changes resulting from the review.

Incomplete stories must return to the product backlog and be reconsidered during the next sprint. They should not automatically be treated as completed work.

4. Sprint Retrospective

After each sprint review, discuss how the team worked together.
Address:

- What went well?
- What caused difficulty?
- What should the team change?
- What specific improvement will the team try during the next sprint?

Document at least one measurable process improvement for the next sprint.

Required Deliverables

1. Project link
    - GitHub repository URL.
    - GitHub Projects or Jira board URL.
2. Product documentation
    - Product name and goal.
    - Intended users.
    - Major features.
    - Setup and usage instructions.
3. Agile evidence
    - Product backlog.
    - Sprint goals.
    - User stories and acceptance criteria.
    - Task assignments and estimates.
    - Scrum meeting records.
    - Sprint review notes.
    - Sprint retrospective notes.
4. Software increment
    - Working source code.
    - Tests or documented test results.
    - Pull requests and peer reviews.
    - Release, deployment, or demonstration instructions.
5. Individual reflection

    Each student must submit a 300–500-word reflection that explains:
    - Their role and technical contributions.
    - How the team used Scrum.
    - An obstacle they encountered and how it was addressed.
    - How project-board data influenced the team's decisions.
    - One lesson they would apply to a future software project.

Object-Oriented Design and Implementation

Each team must design and implement its system using an object-oriented approach.

The team must:

- Use at least one object-oriented programming language.
- Apply fundamental object-oriented principles, including abstraction, encapsulation, inheritance, and polymorphism where appropriate.
- Organize the system into classes with clear responsibilities and relationships.
- Apply at least one appropriate design pattern and explain why it was selected.
- Produce the UML diagrams necessary to communicate the system's design, such as class, sequence, use-case, or state diagrams.
- Maintain concise documentation describing the system architecture, major classes, design decisions, and applied design patterns.
- Demonstrate how the design supports maintainability, changeability, and future extension.

The final submission must include the UML diagrams, design documentation, and a brief explanation identifying where object-oriented principles and design patterns appear in the implementation.

Team Accountability

Every student is expected to participate in planning, meetings, development, review, and documentation. Individual scores may be adjusted based on repository history, project-board activity, meeting records, peer feedback, and the individual reflection. Teams should notify the instructor promptly when participation problems cannot be resolved internally.